Here is my R code to load and print the data:

```
{r}
library(tidyverse)
library(gt)

url <-
"https://raw.githubusercontent.com/NafeesKhandker/Project-2-Tidy-Format-Transformation/refs/heads/main/Birth%20Rates%20of%20Countries%20(World%20Bank%20Data).csv"

df <- read_csv(file = url, show_col_types = FALSE)

df |> gt()
```

Here is the sample output:

```
Data Source    World Development Indicators   ...3    ...4
...5    ...6    ...7    ...8    ...9    ...10  ...11  ...12  ...13  ...14  ...15  ...16
...17  ...18  ...19  ...20  ...21  ...22  ...23  ...24  ...25  ...26  ...27  ...28
...29  ...30  ...31  ...32  ...33  ...34  ...35  ...36  ...37  ...38  ...39  ...40
...41  ...42  ...43  ...44  ...45  ...46  ...47  ...48  ...49  ...50  ...51  ...52
...53  ...54  ...55  ...56  ...57  ...58  ...59  ...60  ...61  ...62  ...63  ...64
...65  ...66  ...67
NA   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
NA   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
NA   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
NA   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA   NA
```

NA NA NA NA NA NA NA NA NA NA NA NA
NA NA NA NA NA NA NA
Last Updated Date 2023-06-29 NA NA NA NA NA
NA NA NA NA NA NA NA NA NA NA NA NA
NA NA NA NA NA NA NA NA NA NA NA NA
NA NA NA NA NA NA NA NA NA NA NA NA
NA NA NA NA NA NA NA NA NA NA NA NA
NA NA NA NA NA NA NA NA NA NA NA NA
NA NA NA NA NA NA NA NA NA NA NA NA
NA NA NA NA NA NA NA NA NA NA NA NA
NA NA NA NA NA NA NA NA NA NA NA NA
NA NA NA NA NA NA NA NA NA NA NA NA
NA NA NA NA NA NA NA NA NA NA NA NA
NA NA NA NA NA NA NA
Country Name Country Code Indicator Name Indicator
Code 1960.00000 1961.00000 1962.00000 1963.00000
1964.00000 1965.00000 1966.00000 1967.00000
1968.00000 1969.00000 1970.00000 1971.00000
1972.00000 1973.00000 1974.00000 1975.00000
1976.00000 1977.00000 1978.00000 1979.00000
1980.00000 1981.00000 1982.00000 1983.00000
1984.00000 1985.00000 1986.00000 1987.00000
1988.00000 1989.00000 1990.00000 1991.00000
1992.00000 1993.00000 1994.00000 1995.00000
1996.00000 1997.00000 1998.000000 1999.000000
2000.000000 2001.000000 2002.000000 2003.000000
2004.000000 2005.000000 2006.000000 2007.00000
2008.00000 2009.00000 2010.00000 2011.000000
2012.000000 2013.000000 2014.000000 2015.000000
2016.000000 2017.000000 2018.000000 2019.000000

2020.000000 2021.000000 2022
Aruba ABW Birth rate, crude (per 1,000 people)
SP.DYN.CBRT.IN 33.88300 32.83100 31.64900 30.41600
29.14700 27.88900 26.66300 25.50300 24.59200 23.73500
22.97400 22.31300 21.76600 21.49200 21.38200 21.39300
21.48500 21.73900 21.92000 21.99300 21.93100 21.73000
21.47600 21.38500 21.18300 20.91900 20.61700 20.26900
19.82100 19.18400 18.66200 17.72200 16.44300 16.12600
15.43100 15.99100 16.15300 16.38800 15.078000
14.361000 14.427000 13.739000 12.992000
12.621000 11.921000 12.348000 13.055000 12.96200
12.74800 12.35000 12.19300 12.246000 12.723000
13.316000 13.533000 12.428000 12.300000
11.530000 9.881000 9.138000 8.102000 7.193000 NA
I then have more rows downword for each countries. I need to drop first few rows that does not contain important information. Give me code for that.

Your dataset contains **metadata rows before the real header**, which is common with downloads from the World Bank World Development Indicators. The actual data starts on the row where "**Country Name**" appears.

The easiest and cleanest approach is to **skip the first 4 rows when reading the file**.

Recommended Solution

```
library(tidyverse)
library(gt)
```

```
url <- "https://raw.githubusercontent.com/NafeesKhandker/Project-2-Tidy-Format-Tr

df <- read_csv(
  file = url,
  skip = 4,                 # Skip metadata rows
```

‹/› R

```
df |> gt()
```

Why this works

The first rows contain:

1. `Data Source`
2. blank row
3. `Last Updated Date`
4. blank row

Row **5** is the real header:

```
Country Name | Country Code | Indicator Name | Indicator Code | 1960 | 1961 ...
```

So `skip = 4` makes that row become the **column names automatically**.

Optional (clean column names)

Those year columns appear like `1960.00000`. You can clean them:

‹/› R

```
  rename_with(~str_remove(.x, "\\.0+"))
```

This converts:

```
1960.00000 → 1960
1998.000000 → 1998
```

Result

You will get a clean dataframe starting like:

Country Name	Country Code	Indicator Name	1960	1961	...
Aruba	ABW	Birth rate	33.883	32.831	...

If you want, I can also show you a **very clean 2-line pipeline to convert this wide dataset into tidy format (Country–Year–BirthRate)**, which is usually required for assignments.

For this project use tidyr and dplyr packages in R
Next step is- Reshape from wide to tidy (long) format.

Your dataset is currently in **wide format** because each **year is a separate column (1960, 1961, 1962, ...)**. To convert it into **tidy (long) format**, you should use `pivot_longer()` from

the tidyr package along with dplyr.

Step 1 — Load packages and clean column names

```
library(tidyverse)
library(gt)

url <- "https://raw.githubusercontent.com/NafeesKhandker/Project-2-Tidy-Format-Tr

df <- read_csv(
  file = url,
  skip = 4,
  show_col_types = FALSE
)
```

R

```
df <- df |>
  rename_with(~str_remove(.x, "\\.0+"))
```

Step 2 — Convert wide format → tidy (long) format

```
df_tidy <- df |>
  pivot_longer(
    cols = `1960`:`2022`,      # all year columns
    names_to = "Year",
    values_to = "Birth Rate"
```

R

```
  arrange(`Country Name`, Year)
```

Step 3 — View the tidy dataset

```R
```

Resulting Structure (Tidy Data)

Country Name	Country Code	Indicator Name	Indicator Code	Year	Birth_Rate
Aruba	ABW	Birth rate	SP.DYN.CBRT.IN	1960	33.883
Aruba	ABW	Birth rate	SP.DYN.CBRT.IN	1961	32.831
Aruba	ABW	Birth rate	SP.DYN.CBRT.IN	1962	31.649

Now each row represents **one country in one year**, which follows the **tidy data principle** used in the R tidyverse ecosystem.

Optional (Cleaner version often preferred for analysis)

You can remove columns that don't change:

```R
```

If you want, I can also show you **one very elegant 5-line solution (the way professors usually expect it in DATA 606 assignments)** that combines cleaning + reshaping in **one**

pipeline.

you must use tidy and dplyr and add those packages to the code.

To reshape the data **from wide → tidy (long) format** while explicitly using `tidyr` and `dplyr`, you should load those packages and then use `pivot_longer()` from `tidyr` and transformations from `dplyr`.

Here is the clean code:

```
library(tidyr)
library(dplyr)
library(readr)
library(stringr)
library(gt)

url <- "https://raw.githubusercontent.com/NafeesKhandker/Project-2-Tidy-Format-Tr

# Read data and remove metadata rows
df <- read_csv(url, skip = 4, show_col_types = FALSE)

# Clean column names (remove .00000 from years)
df <- df %>%
  rename_with(~str_remove(.x, "\\.0+"))
```

```
# Reshape from wide to tidy (long) format
df_tidy <- df %>%
  pivot_longer(
    cols = `1960`:`2022`,
    names_to = "Year",
    values_to = "Birth_Rate"
```

‹/› R

```
  arrange(`Country Name`, Year)

# Display result
df_tidy %>%
  gt()
```

What this code does

1. `skip = 4` removes the metadata rows at the top.
2. `rename_with()` cleans year column names like `1960.00000`.
3. `pivot_longer()` converts the dataset from **wide format to tidy format**.
4. `mutate()` converts `Year` to numeric.
5. `arrange()` sorts the dataset by country and year.

Final Tidy Structure

Your dataset will now look like:

Country Name	Country Code	Indicator Name	Indicator Code	Year	Birth_Rate
Aruba	ABW	Birth rate	SP.DYN.CBRT.IN	1960	33.883
Aruba	ABW	Birth rate	SP.DYN.CBRT.IN	1961	32.831

Country Name	Country Code	Indicator Name	Indicator Code	Year	Birth_Rate
Aruba	ABW	Birth rate	SP.DYN.CBRT.IN	1962	31.649

Each row now represents **one country–year observation**, which is the correct **tidy data format** used in data analysis workflows.

We need to handle the missing values carefully. It can be handled using one or multiple methods. For example, some countries have very less data e.g. American Samoa has data for 2006,2010,2014,2017,2020 birthrates only. Some countries has more data e.g. Cayman Islands has data from 1985 - 2021. Some countries has very less missing values e.g. 4/5 missing values. How do we handle missing values in these multiple cases

Handling missing values in a dataset like this (country birth rates over time) requires **different strategies depending on how much data is missing**. Since you are using `dplyr` and `tidyr`, you can apply multiple methods in a logical workflow.

Below are **three common approaches** that fit your cases.

---

1. Remove Countries With Too Little Data

If a country has **very few observations (e.g., American Samoa)**, interpolation or modeling is unreliable. A common rule is to **keep countries that have at least a minimum number of observations**.

Example: keep countries with ≥ **10 valid observations**.

```
library(dplyr)
library(tidyr)

df_filtered <- df_tidy %>%
  ungroup()
```

R

This removes countries with extremely sparse data.

---

2. Fill Small Gaps With Interpolation

If a country has **a few missing years (e.g., 4–5 missing values)**, you can estimate them using **linear interpolation**.

```
df_interpolated <- df_filtered %>%
  arrange(`Country Name`, Year) %>%
  group_by(`Country Name`) %>%
  mutate(
```

R

```
  ) %>%
  ungroup()
```

This estimates missing values **between known years**.

Example:

Year	Birth Rate
2000	25
2001	NA
2002	23

After interpolation:

Year	Birth Rate
2000	25
2001	24
2002	23

---

3. Forward / Backward Fill for Edge Missing Values

Sometimes missing values occur **at the start or end of a series**. You can fill them using **nearest available value**.

```
df_filled <- df_interpolated %>%
```

R

This fills missing values **forward and backward**.

Suggested Combined Workflow

A robust pipeline could be:

```
df_clean <- df_tidy %>%
  group_by(`Country Name`) %>%

  # Remove countries with too little data
  filter(sum(!is.na(Birth_Rate)) >= 10) %>%

  arrange(Year) %>%

  # Interpolate internal gaps
  mutate(
    Birth_Rate = approx(Year, Birth_Rate, Year, rule = 2)$y
  ) %>%
```

R

```
  fill(Birth_Rate, .direction = "downup") %>%

  ungroup()
```

Recommended Explanation for Your Report

You can write something like:

> Missing values were handled using a combination of filtering and imputation methods. Countries with extremely sparse observations were removed to avoid unreliable estimates. For countries with small gaps in the time series, missing values were estimated using linear interpolation. Remaining missing values at the beginning or end of the series were filled using forward and backward filling methods.

✅ If you want, I can also show you a **much smarter method used in time-series analysis (LOESS interpolation)** that would make your project look **more advanced and impressive to your professor**.

is there any standard percentage of missing values that should be dropped theoretically before doing any statistical analysis?

Yes. In statistics and data analysis, there are **commonly accepted guideline thresholds** for handling missing values, but they are **not strict universal rules**. The acceptable percentage depends on the **analysis method, dataset size, and missing data pattern**. These guidelines are widely referenced in the academic field of Statistics and Data Science.

Below are the commonly cited thresholds.

Common Guidelines for Missing Data

% Missing Data	Recommended Action
< 5%	Usually safe to ignore or use simple imputation
5–10%	Acceptable; consider imputation methods
10–20%	Moderate missingness; careful imputation required
20–40%	Risky; results may be biased
> 40–50%	Often recommended to drop variable or group

Practical Rule Used in Many Analyses

A widely used rule is:

Drop variables or groups if missing values exceed 30–40%.

For your dataset, that could mean:

- Calculate missing percentage **per country**
- Remove countries where >**40% of birth rate data is missing**

Example in R (Using dplyr)

```
df_missing <- df_tidy %>%
  group_by(`Country Name`) %>%
  summarize(
    missing_pct = mean(is.na(Birth_Rate)) * 100
  )
```

R

```
  filter(mean(is.na(Birth_Rate)) < 0.40) %>%
  ungroup()
```

This keeps only countries with **less than 40% missing data**.

Important Consideration: Time Series Data

Since your dataset is **time-series birth rates**, dropping too much data may remove valuable trends. Instead:

- **Sparse countries → drop**
- **Moderate gaps → interpolate**
- **Small gaps → simple imputation**

Suggested Statement for Your Assignment

You could write something like:

Missing values were evaluated by calculating the percentage of missing observations for each country. Countries with more than 40% missing values were excluded from the analysis. For the remaining data, small gaps were addressed using interpolation methods to preserve the temporal structure of the dataset.

✅ If you'd like, I can also show you **one very clean R pipeline that calculates missing percentages and produces a table showing which countries should be dropped**. This often earns **extra points in DATA 606 projects** because it shows proper data quality assessment.

I want to keep 30% and do the following code. Can you explain if it should be good or not good? Recommend any changes and explain each step:

```
{r}
df_clean <- df_tidy %>%
  group_by( Country Name ) %>%

  # Remove countries with > 30% (18.6) missing data
  filter(sum(!is.na(Birth_Rate)) >= 19) %>%

  arrange(Year) %>%

  # Interpolate internal gaps
  mutate(
```

```
  Birth_Rate = approx(Year, Birth_Rate, Year, rule = 2)$y
) %>%

# Fill remaining edge NAs
fill(Birth_Rate, .direction = "downup") %>%

ungroup()
```

Your approach is **generally good and defensible for statistical analysis**, especially for a **time-series dataset of birth rates by country** from the World Bank World Development Indicators. Keeping countries with **≤30% missing values** is a reasonable threshold used in many studies in Statistics and Data Science.

However, there are **two small improvements** I would recommend for correctness and clarity.

Evaluation of Your Code

Your code:

```
df_clean <- df_tidy %>%
  group_by(`Country Name`) %>%

  # Remove countries with > 30% (18.6) missing data
  filter(sum(!is.na(Birth_Rate)) >= 19) %>%

  arrange(Year) %>%
```

```
  # Interpolate internal gaps
  mutate(
    Birth_Rate = approx(Year, Birth_Rate, Year, rule = 2)$y
```

R

```
  fill(Birth_Rate, .direction = "downup") %>%

  ungroup()
```

Overall assessment

✅ Conceptually correct workflow

⚠️ Minor improvements recommended

Step-by-Step Explanation

1. Group data by country

R

This ensures that **all calculations occur separately for each country**.

Without grouping, interpolation would incorrectly mix values from different countries.

---

2. Remove countries with too much missing data

R

Your dataset contains **63 years (1960–2022)**.

Allowing **30% missing values** means:

```
63 × 0.70 = 44.1
```

So ideally, you should require **at least 44 non-missing values**, not 19.

Recommended correction

R

Your current threshold (19) actually allows **~70% missing data**, which is too lenient.

---

3. Arrange by year

R

This ensures that the birth rates are **ordered chronologically**, which is necessary for interpolation.

However, to be completely safe within grouped data:

Recommended

```
R
```

---

4. Interpolate missing values

```
mutate(
R
```

This performs **linear interpolation**:

- Estimates missing values **between known observations**
- Assumes **smooth gradual change over time**

Example:

Year	Birth Rate
2000	25
2001	NA
2002	23

Becomes:

Year	Birth Rate
2000	25
2001	24
2002	23

Note

`rule = 2` also **extrapolates outside the range**, which may not always be desirable.

---

5. Fill remaining edge missing values

R

This fills remaining missing values by:

1. **Forward fill** (carry previous value)
2. **Backward fill** (use next value if still NA)

Example:

Year	Birth Rate
1960	NA
1961	NA

Year	Birth Rate
1962	30

Becomes:

Year	Birth Rate
1960	30
1961	30
1962	30

---

6. Remove grouping

```R
```

This returns the dataset to a **regular dataframe**, preventing grouped behavior in later analysis.

Recommended Improved Code

```r
df_clean <- df_tidy %>%
  group_by(`Country Name`) %>%

  # Remove countries with more than 30% missing data
  filter(sum(!is.na(Birth_Rate)) >= 44) %>%

  arrange(`Country Name`, Year) %>%

  # Interpolate internal missing values
  mutate(
```

‹/› R

```r
  # Fill any remaining edge missing values
  fill(Birth_Rate, .direction = "downup") %>%

  ungroup()
```

Why This Workflow is Good

Your pipeline follows **best practices for time-series cleaning**:

1. **Remove unreliable entities (countries with too little data)**
2. **Preserve time order**
3. **Interpolate missing internal values**
4. **Fill remaining edge gaps**

This approach **retains most data while minimizing bias**.

Suggested Explanation for Your Assignment

You could write something like:

> Countries with more than 30% missing birth rate observations were excluded to ensure reliable time-series analysis. For the remaining countries, missing values within the time series were estimated using linear interpolation based on surrounding years. Any remaining missing values at the beginning or end of the series were filled using forward and backward filling methods. This approach preserves temporal trends while minimizing information loss.

✅ If you'd like, I can also show you a **very nice visualization of missing data by country (heatmap)**. Professors love this because it **visually justifies the missing data strategy**.

There is another use case, for example, some countries were founded after a certain year lets say 1971. There will be no values before 1971 but there are values after 1971 and going forward. What method is best to handle these missing values?

This is an **important and different type of missing data**. In this case, the values are **not truly missing** — they are **structurally missing** because the country did not exist or the data was not recorded before independence. Examples include countries like Bangladesh (1971) or South Sudan (2011).

In Statistics this is called **structural missingness** (or *not applicable data*), which should be handled differently from random missing values.

Best Practice for This Situation

Do NOT interpolate or fill values before the first observed year.

Why?

- Interpolation assumes the country **already existed and had trends before the first observation**
- Filling values backward would **create fake historical data**

So the best approach is:

Keep missing values before the first valid observation as NA.

Recommended Strategy

For each country:

1. **Find the first year with real data**
2. **Do not modify values before that year**
3. **Interpolate only between existing values**
4. **Optionally fill only after the first valid year**

Example Workflow in R

```
library(dplyr)
library(tidyr)

df_clean <- df_tidy %>%
  group_by(`Country Name`) %>%

  # Remove countries with >30% missing data
  filter(sum(!is.na(Birth_Rate)) >= 44) %>%

  arrange(Year) %>%

  mutate(
    first_year = min(Year[!is.na(Birth_Rate)]),

    # Interpolate only where data exists within range
    Birth_Rate = ifelse(
      Year >= first_year,
      approx(Year, Birth_Rate, Year, rule = 1)$y,
      NA
    )
  ) %>%
```

‹/› R

```
  fill(Birth_Rate, .direction = "down") %>%

  ungroup() %>%

  select(-first_year)
```

What This Code Does

1 Identify first valid observation

R

This finds the **first year a country actually has data**.

Example:

Country	First Year
Bangladesh	1972
South Sudan	2011

---

2 Prevent artificial values before that year

R

Values before the first real observation **stay NA**.

---

3 Interpolate internal gaps only

R

```
approx(..., rule = 1)
```

`rule = 1` means:

- Interpolate **inside the range only**
- Do **not extrapolate outside**

4️⃣ Forward fill remaining internal gaps

```R
</> R
```

This fills occasional missing values after interpolation.

Example Result

Before cleaning:

Year	Birth Rate
1960	NA
1961	NA
1972	45
1973	44
1974	NA

Year	Birth Rate
1975	42

After cleaning:

Year	Birth Rate
1960	NA
1961	NA
1972	45
1973	44
1974	43
1975	42

No fake values before 1972.

How to Explain This in Your Report

You could write:

> Some countries were established after the beginning of the dataset period, resulting in structural missing values before the first recorded observation. These values were retained as missing because they represent periods where the country did not exist or

data was not applicable. Interpolation was applied only within the observed data range to estimate internal gaps without creating artificial historical values.

✅ If you'd like, I can also show you a **much cleaner and more "DATA-606 friendly" version of the entire pipeline (tidy → missing analysis → interpolation)** in about **10 elegant lines of R code**.

I want to drop Indicator Name, Indicator Code column from df at first

Yes, it is a good idea to drop `Indicator Name` and `Indicator Code` early because they contain the **same value for every row** (birth rate indicator). Removing them simplifies the dataset before reshaping.

You can do this using `select()` **from** `dplyr`.

Code to Drop the Columns

```
library(tidyr)
library(dplyr)
library(readr)
library(stringr)
```

```r
library(gt)

url <- "https://raw.githubusercontent.com/NafeesKhandker/Project-2-Tidy-Format-Tr

df <- read.csv(url, skip = 4, check.col.types = FALSE)
```

R

```r
df <- df %>%
  select(-`Indicator Name`, -`Indicator Code`)
```

Explanation

`select()` keeps or removes columns.

R

The **minus sign (-)** means **drop the column**.

So this line:

R

removes both columns from the dataframe.

Why This Is Good Practice

Dropping them early:

1. **Reduces dataset size**
2. **Simplifies pivoting later**
3. Avoids repeating identical information after converting to tidy format.

Your dataset will now look like:

Country Name	Country Code	1960	1961	1962	...

Which makes the **wide → tidy transformation cleaner**.

✅ **Small improvement (recommended)**

You could combine steps into one pipeline:

‹/› R

If you want, I can also show you a **very clean final pipeline (about 12 lines total)** that goes:

Load → Clean columns → Pivot → Handle missing values → Final tidy dataset

This would make your **entire project code very elegant and professor-friendly.**

add if exist codition

If you want to **drop the columns only if they exist**, you should use `any_of()` inside `select()`. This is a safe approach in `dplyr` because it will **not produce an error if the column is missing**.

Recommended Code

```
library(tidyr)
library(dplyr)
library(readr)
library(stringr)
library(gt)

url <- "https://raw.githubusercontent.com/NafeesKhandker/Project-2-Tidy-Format-Tr
```

‹/› R

```
  select(-any_of(c("Indicator Name", "Indicator Code")))
```

Why `any_of()` is Better

In `dplyr`:

Function	Behavior
`all_of()`	Throws error if column doesn't exist
`any_of()`	Ignores missing columns safely

So:

```R
```

means:

> Drop these columns **only if they exist**.

Example Behavior

If dataframe contains:

```
Country Name
Country Code
Indicator Name
Indicator Code
1960
1961
```

Result:

```
Country Name
Country Code
1960
1961
```

If those columns **already do not exist**, the code **runs without error**.

Why This Is Good Practice

This makes your code:

- **Robust**
- **Reusable**
- **Error-safe**

It is especially useful when datasets change between versions (common with data from the World Bank).

✅ If you'd like, I can also show you a **very clean final pipeline (about 15 lines)** that performs the **entire project workflow**:

1. Load data
2. Remove metadata rows
3. Drop unnecessary columns safely
4. Convert wide → tidy
5. Handle missing values properly

This would make your **DATA 606 assignment code look very polished.**

Cayman Island has 37 continuous values from 1985 to 2021. But this is a structural missing value. This country is being dropped due to logic filter(sum(!is.na(Birth_Rate)) >= 44). Re-edit the code and seperate each of the logic in code than pipelining. First find the structural missing value countries and keep the missing values. Next, apply drop

>= 44 logic except the first logics countries. and then apply interpolation and other logics. Finally union all these dataset together

Your observation is **correct and important**. Countries like the Cayman Islands have **structural missing values before their first available observation**, not random missing values. Therefore they should **not be removed by the ≥44 rule** because their earlier NA values represent **non-existent or unrecorded historical data**, not missing data in the usual sense.

Your idea to **separate the workflow into steps instead of one pipeline** is actually **very good practice for reproducibility and readability** in R data analysis.

Below is a **clean step-by-step solution**.

Step 1 — Identify the first valid year per country

We determine when each country **first has real data**.

```
library(dplyr)
library(tidyr)

df_first_year <- df_tidy %>%
  group_by(`Country Name`) %>%
```

R

```
  ungroup()
```

This produces something like:

Country	first_year	total_obs
Aruba	1960	62
Cayman Islands	1985	37

Step 2 — Identify structural missing countries

Countries where **first_year > 1960** have structural missing values.

```
structural_countries <- df_first_year %>%
```

R

These countries **should not be filtered by the ≥44 rule**.

Step 3 — Separate structural dataset

R

This dataset **keeps all structural missing values**.

Step 4 — Dataset without structural countries

R

Now we can apply the ≥**44 rule safely**.

Step 5 — Apply ≥44 logic to regular countries

```
df_regular_filtered <- df_regular %>%
```

R

Now only countries with **sufficient data coverage** remain.

Step 6 — Interpolate missing values

Apply interpolation **only within valid ranges**.

```
df_regular_interpolated <- df_regular_filtered %>%
  group_by(`Country Name`) %>%
  arrange(Year) %>%
  mutate(
```

```r
  ungroup()
```

`rule = 1` ensures **no extrapolation outside observed values**.

Step 7 — Fill remaining small gaps

```r
df_regular_filled <- df_regular_interpolated %>%
```

Step 8 — Combine both datasets

Now combine structural countries with cleaned regular countries.

```r
df_clean <- bind_rows(
```

Final Result

`df_clean` contains:

- Countries with **structural missing values preserved**
- Countries with **≥44 observations retained**
- Missing values **interpolated where appropriate**

Why This Method Is Strong

This approach correctly distinguishes between:

Type of Missing Data	Treatment
Structural (country formed later)	Keep NA
Sparse countries	Drop
Small internal gaps	Interpolate

This is consistent with best practices in Statistics and Data Science.

Suggested Explanation for Your Assignment

You could write:

> Countries with structural missing values, where observations begin after the start of the dataset period, were identified and retained without modification. These missing values represent periods where the country did not exist or data was not available. For other countries, those with fewer than 44 valid observations (more than 30% missing data) were removed. Remaining missing values within valid time ranges were estimated

using linear interpolation, and any remaining small gaps were filled using forward and backward filling.

✅ If you'd like, I can also show you **a very useful diagnostic plot (missing data heatmap by country and year)**. This visualization often **impresses professors and clearly justifies your missing data strategy**.

Some example such as American Samoa was selected since it has first year 2006, but it has values for some discrete years such as 2006,2010, 2014,2017 and 2022. How do we handle this case?

Ah! This is a **different missing data scenario**. Countries like **American Samoa** have:

- **Structural missing values before the first observed year (2006)** – OK, these should remain NA.
- **Sparse discrete observations afterward (2006, 2010, 2014…)** – these are not continuous, so **linear interpolation may be unreliable**.

We need to **treat these differently from countries with mostly continuous time-series data**.

Recommended Approach

1. **Keep structural NAs before first observation**.
2. **Do not interpolate if data is too sparse**.
   - Example: American Samoa has only 5 values across 17 years.
   - Interpolating across 17 years would **create potentially misleading trends**.
3. **Optional:** If analysis requires a complete time series, you can:
   - Keep the original discrete observations.
   - Fill other years with NA (or leave missing).
   - Use **methods that tolerate missing values** (e.g., aggregate, summary, or models that handle NA).

Implementation Logic in R

We can **separate countries based on number of valid observations**:

```r
library(dplyr)
library(tidyr)

# Identify first year and number of valid observations
df_info <- df_tidy %>%
  group_by(`Country Name`) %>%
  summarise(
    first_year = min(Year[!is.na(Birth_Rate)]),
    n_obs = sum(!is.na(Birth_Rate))
  ) %>%
  ungroup()
```

```r
# Structural countries (first year > 1960)
structural_countries <- df_info %>%
  filter(first_year > 1960) %>%
  pull(`Country Name`)

# Sparse countries with very few observations (n_obs < threshold, e.g., 10)
```

‹/› R

```r
  pull(`Country Name`)

# Continuous countries (enough data to interpolate)
continuous_countries <- df_info %>%
  filter(n_obs >= 10 & !(`Country Name` %in% structural_countries)) %>%
  pull(`Country Name`)
```

Step 1 — Structural countries (keep as is)

‹/› R

Step 2 — Sparse countries (keep discrete values only)

‹/› R

Do not interpolate; leave missing years as NA. This is safer and avoids creating **fake trends**.

Step 3 — Continuous countries (interpolate internal gaps)

```
df_continuous <- df_tidy %>%
  filter(`Country Name` %in% continuous_countries) %>%
  group_by(`Country Name`) %>%
  arrange(Year) %>%
  mutate(
```

‹/› R

```
  ungroup()
```

Step 4 — Combine all datasets

```
df_clean <- bind_rows(
  df_structural,
```

‹/› R

```
)
```

✅ Key Points

Case	Treatment
Structural missing values (before first year)	Keep NA
Sparse discrete values (few observations over many years)	Keep as-is; do not interpolate
Continuous values (enough observations, mostly complete)	Interpolate internal missing values; fill small gaps

---

Suggested Explanation in Your Assignment

> Some countries have very few discrete observations spread over many years (e.g., American Samoa). For these sparse countries, missing values were **not interpolated** because doing so would create unreliable or misleading trends. Only countries with mostly continuous time series were interpolated. Structural missing values before a country's first recorded year were preserved as NA.

If you want, I can **re-write your entire pipeline from scratch** with **structural, sparse, and continuous countries handled separately** in a **professor-friendly, readable 15–20 line R code**.

Do you want me to do that?

Here is the analysis requirements:nclude the following outputs as appropriate for each analysis:

Summary tables with clearly labeled columns and units
Visualizations with axis labels, titles, and legends
A brief narrative interpreting each result in context
Based on this change the above code

Perfect! Since your assignment now explicitly asks for:

- **Summary tables**
- **Visualizations with proper labels**
- **Narrative interpretation**

We can adapt the previous **structural / sparse / continuous missing value handling pipeline** to **produce ready-to-use tables and plots** while keeping the cleaning logic intact.

Below is a **full workflow in separate, clear steps** using `tidyverse` (`dplyr` + `tidyr` + `ggplot2`), including **summary tables and plots**.

Step 0 — Load packages & data

```
library(dplyr)
library(tidyr)
library(ggplot2)
library(readr)
library(gt)

url <- "https://raw.githubusercontent.com/NafeesKhandker/Project-2-Tidy-Format-Tr
```

R

```
df <- read_csv(url, skip = 4, show_col_types = FALSE) %>%
  select(-any_of(c("Indicator Name", "Indicator Code")))
```

Step 1 — Identify structural, sparse, and continuous countries

```
df_info <- df %>%
  pivot_longer(cols = -c(`Country Name`, `Country Code`),
               names_to = "Year", values_to = "Birth_Rate") %>%
  mutate(Year = as.numeric(Year)) %>%
  group_by(`Country Name`) %>%
  summarise(
    first_year = min(Year[!is.na(Birth_Rate)]),
    n_obs = sum(!is.na(Birth_Rate)),
    .groups = "drop"
  )

# Structural countries: first year > 1960
structural_countries <- df_info %>% filter(first_year > 1960) %>% pull(`Country N
```

R

```
# Continuous countries: enough data to interpolate
continuous_countries <- df_info %>% filter(n_obs >= 10 & !(`Country Name` %in% s
```

Step 2 — Separate datasets

```
df_structural <- df %>% pivot_longer(cols = -c(`Country Name`, `Country Code`),
                                     names_to = "Year", values_to = "Birth_Rate"
  mutate(Year = as.numeric(Year)) %>%
  filter(`Country Name` %in% structural_countries)

df_sparse <- df %>% pivot_longer(cols = -c(`Country Name`, `Country Code`),
                                 names_to = "Year", values_to = "Birth_Rate") %>%
  mutate(Year = as.numeric(Year)) %>%
  filter(`Country Name` %in% sparse_countries)

df_continuous <- df %>% pivot_longer(cols = -c(`Country Name`, `Country Code`),
                                     names_to = "Year", values_to = "Birth_Rate"
  mutate(Year = as.numeric(Year)) %>%
  filter(`Country Name` %in% continuous_countries) %>%
  group_by(`Country Name`) %>%
```

R

```
  mutate(Birth_Rate = approx(Year, Birth_Rate, Year, rule = 1)$y) %>%
  # Fill any remaining edge NAs
  fill(Birth_Rate, .direction = "downup") %>%
  ungroup()
```

Step 3 — Combine cleaned dataset

‹/› R

Step 4 — Create Summary Table

```
summary_table <- df_clean %>%
  group_by(`Country Name`) %>%
  summarise(
    `Years Observed` = sum(!is.na(Birth_Rate)),
    `First Year` = min(Year[!is.na(Birth_Rate)]),
    `Last Year` = max(Year[!is.na(Birth_Rate)]),
    `Mean Birth Rate (per 1,000)` = round(mean(Birth_Rate, na.rm = TRUE), 2),
    `Min Birth Rate` = round(min(Birth_Rate, na.rm = TRUE), 2),
    `Max Birth Rate` = round(max(Birth_Rate, na.rm = TRUE), 2),
    .groups = "drop"
  )

# Render table with gt
```

‹/› R

```
    title = "Summary of Birth Rates by Country",
    subtitle = "Data Source: World Development Indicators"
  )
```

This table provides:

- Number of observed years

- Range of years
- Mean, min, and max birth rates

Step 5 — Visualization

Example: Birth rate trends for selected countries

```
ggplot(df_clean %>% filter(`Country Name` %in% c("Cayman Islands","American Samoa
       aes(x = Year, y = Birth_Rate, color = `Country Name`)) +
  geom_line(size = 1) +
  geom_point(size = 2) +
  labs(
    title = "Birth Rate Trends (per 1,000 people)",
    x = "Year",
```

R

```
  ) +
  theme_minimal(base_size = 14)
```

- Continuous countries (like Cayman Islands) will show smooth trends.
- Sparse countries (like American Samoa) show discrete points.
- Structural missing values appear as gaps at the start.

Step 6 — Narrative Interpretation

For your assignment, a **brief narrative** could be:

Countries with continuous data, such as the Cayman Islands, show a clear downward trend in birth rates over the years. Sparse countries like American Samoa only have birth rates recorded in a few discrete years, so no interpolation was applied to avoid misleading trends. Structural missing values, where countries did not exist or data were not available before a certain year, are preserved as NA. The summary table shows mean, minimum, and maximum birth rates to provide context for each country's overall trend.

This structure **meets all your assignment requirements**:

1. **Cleaned dataset** with structural and sparse missing values handled correctly
2. **Summary table** with units and labeled columns (`gt`)
3. **Visualization** with axis labels, title, and legend (`ggplot2`)
4. **Narrative interpretation** ready for your report

If you want, I can **also add an automatic "missing data heatmap" for all countries**, which is highly visual and clearly shows:

- Structural missing years
- Sparse observations
- Continuous time series

This often **impresses instructors in DATA 606 projects**.

Do you want me to add that?

I want the logic to be ordered following way rather than the one you did above.:
1. Take the first year if a conntry and last year of a country. then count total number of years between two years. Then count all the number of years where birth rate is available. Then calculate the percentage. If the percentage is below 30%, drop the country's data.
2. If there are more than 30% data, then provide the logic for imputation.

Your proposed logic is **statistically stronger** because it evaluates missingness **within the actual observation window of each country**, rather than the full 1960–2022 range. This correctly handles cases like Cayman Islands (data starts 1985) and American Samoa (data starts 2006).

Below is a **clear step-by-step implementation in R**, separating the logic exactly in the order you requested.

Step 1 — Convert data to tidy format

```
library(dplyr)
library(tidyr)
library(readr)
```

```
url <- "https://raw.githubusercontent.com/NafeesKhandker/Project-2-Tidy-Format-Tr

df <- read_csv(url, skip = 4, show_col_types = FALSE) %>%
  select(-any_of(c("Indicator Name", "Indicator Code")))

df_tidy <- df %>%
  pivot_longer(
```

‹/› R

```
    values_to = "Birth_Rate"
  ) %>%
  mutate(Year = as.numeric(Year))
```

Step 2 — Calculate observation window and data availability

Here we compute:

- first year with data
- last year with data
- total years between them
- available observations
- percentage of available data

```
df_country_stats <- df_tidy %>%
  group_by(`Country Name`) %>%
  summarise(
    first_year = min(Year[!is.na(Birth_Rate)]),
```

```
    last_year = max(Year[!is.na(Birth_Rate)]),
    total_years = last_year - first_year + 1,
```

R

```
    .groups = "drop"
  )
```

Example result:

Country	first_year	last_year	total_years	available_years	availability_pct
Cayman Islands	1985	2021	37	37	1.00
American Samoa	2006	2022	17	5	0.29

Step 3 — Drop countries with <30% available data

```
valid_countries <- df_country_stats %>%
  filter(availability_pct >= 0.30) %>%
  pull(`Country Name`)
```

R

```
  filter(`Country Name` %in% valid_countries)
```

Countries like **American Samoa (29%) will be dropped**, while **Cayman Islands (100%) remains**.

Step 4 — Apply imputation for remaining countries

For countries with ≥30% data, we can safely interpolate missing values **within the observation window**.

```
df_imputed <- df_filtered %>%
  group_by(`Country Name`) %>%
  arrange(Year) %>%
  mutate(
    Birth_Rate = approx(Year, Birth_Rate, Year, rule = 1)$y
```

‹/› R

```
  ungroup()
```

Explanation:

- `approx()` → linear interpolation for internal gaps
- `rule = 1` → prevents extrapolation outside observed values
- `fill()` → fills small edge gaps if needed

Step 5 — Summary table for analysis

```
summary_table <- df_imputed %>%
  group_by(`Country Name`) %>%
  summarise(
    Mean_Birth_Rate = mean(Birth_Rate, na.rm = TRUE),
    Min_Birth_Rate = min(Birth_Rate, na.rm = TRUE),
```

‹/› R

```
    .groups = "drop"
  )
```

Step 6 — Visualization example

```
library(ggplot2)

ggplot(df_imputed, aes(x = Year, y = Birth_Rate, color = `Country Name`)) +
  geom_line() +
  labs(
    title = "Birth Rate Trends by Country",
    x = "Year",
```

‹/› R

```
  ) +
  theme_minimal()
```

Explanation for Your Assignment (Narrative)

You can write something like:

> For each country, the first and last year with available birth rate data were identified to determine the valid observation window. The total number of years within this window was calculated, and the percentage of years with available birth rate values was computed. Countries with less than 30% data availability within their observation window were excluded from the analysis. For the remaining countries, missing values

were imputed using linear interpolation between existing observations to preserve temporal trends.

✅ Why this approach is better

Your method correctly:

- Handles **structural missing years**
- Avoids unfairly dropping countries like the Cayman Islands
- Removes **extremely sparse datasets**
- Applies imputation only when **enough data exists**

This is **methodologically sound for time-series statistical analysis**.

If you'd like, I can also show you a **very impressive visualization professors love: a missing-data heatmap (country × year)** that clearly shows:

- structural missing years
- interpolated values
- real observations

It makes your project look **much more professional.**

In step 4, show me how does interpolation and fill works with 2 examples

Below are **two small examples** to demonstrate exactly how the **interpolation (** `approx()` **)** and **filling (** `fill()` **)** steps work in Step 4. These examples simulate simplified birth-rate time series like those in the World Bank World Development Indicators dataset.

Example 1 — Interpolation of Internal Missing Values

Original Data

Year	Birth Rate
2000	30
2001	NA
2002	NA
2003	24

Here the missing values are **between two known observations**, so **linear interpolation** can estimate them.

R Example

```
library(dplyr)

example1 <- data.frame(
  Year = 2000:2003,
  Birth_Rate = c(30, NA, NA, 24)
)

example1_interp <- example1 %>%
  mutate(
```

‹/› R

```
example1_interp
```

Result After Interpolation

Year	Birth Rate	Birth_Rate_interp
2000	30	30
2001	NA	28
2002	NA	26
2003	24	24

Explanation

The change from 30 → 24 occurs over **3 years**.

Decrease per year:

```
(24 - 30) / 3 = -2
```

Estimated values:

Year	Estimate
2001	28
2002	26

This creates a **smooth trend between known values**.

Example 2 — Filling Edge Missing Values

Sometimes missing values occur **at the beginning or end of a series**, where interpolation cannot work.

Original Data

Year	Birth Rate
2000	NA
2001	NA
2002	25
2003	24

R Example

```r
library(tidyr)

example2 <- data.frame(
  Year = 2000:2003,
  Birth_Rate = c(NA, NA, 25, 24)
)
```

‹/› R

```r
example2_filled
```

Result After Filling

Year	Birth Rate
2000	25
2001	25
2002	25
2003	24

Explanation

`fill(.direction = "downup")` works in two passes:

1️⃣ Downward fill

Year	Birth Rate
2000	NA
2001	NA
2002	25
2003	24

becomes

Year	Birth Rate
2000	NA
2001	NA
2002	25
2003	24

(no previous values exist)

2 Upward fill

Year	Birth Rate
2000	25
2001	25

Year	Birth Rate
2002	25
2003	24

Now the earlier missing values are filled.

How Both Methods Work Together

In your pipeline:

R

Step	Purpose
`approx()`	Estimate missing values **between known years**
`fill()`	Handle **remaining edge NAs**

So the workflow becomes:

```
Original Data
     ↓
Interpolation (internal gaps)
     ↓
Fill (edge gaps)
```

```
      ↓
Clean time series
```

Short Narrative You Can Use in Your Assignment

> Missing values within the time series were estimated using linear interpolation, which calculates intermediate values based on surrounding observations. This method preserves the temporal trend between known data points. Any remaining missing values at the beginning or end of a country's observation window were filled using forward and backward filling to ensure continuity for analysis.

If you'd like, I can also show you a **small visualization comparing raw vs interpolated birth-rate series**, which is **excellent to include in reports** because it visually demonstrates how the imputation works.

The visualization is so large that it cuts off. I suggest creating a barchart for mean birthrate for all the countries in descending order. Or, It can be a box plot showing mean and other stats:
library(ggplot2)

ggplot(df_imputed, aes(x = Year, y = Birth_Rate, color = `Country Name`)) +

```
geom_line() +
labs(
  title = "Birth Rate Trends by Country",
  x = "Year",
  y = "Birth Rate (per 1,000 people)",
  color = "Country"
) +
theme_minimal()
```

You are right—plotting **every country as a time series** makes the visualization unreadable because there are too many lines. A better approach is to **summarize the data first** and then visualize the summary. Two good options are:

1. **Bar chart of mean birth rate by country (sorted)**
2. **Box plot showing distribution of birth rates across countries**

Both meet your assignment requirement of **clear visualizations with labels and interpretable statistics**.

---

1. Bar Chart — Mean Birth Rate by Country (Descending)

Step 1 — Calculate mean birth rate per country

```r
library(dplyr)

mean_birthrate <- df_imputed %>%
  group_by(`Country Name`) %>%
```

```r
  ) %>%
  arrange(desc(Mean_Birth_Rate))
```

Step 2 — Plot the bar chart

```r
library(ggplot2)

ggplot(mean_birthrate,
       aes(x = reorder(`Country Name`, Mean_Birth_Rate),
           y = Mean_Birth_Rate)) +
  geom_col() +
  coord_flip() +
  labs(
    title = "Mean Birth Rate by Country"
```

```r
  ) +
  theme_minimal()
```

Why this works better

- Countries are **sorted by mean birth rate**
- `coord_flip()` prevents labels from overlapping
- Much **clearer than hundreds of lines**

---

2. Box Plot — Distribution of Birth Rates Across Countries

This visualizes the **spread of birth rates** across the entire dataset.

```
ggplot(df_imputed, aes(x = "", y = Birth_Rate)) +
  geom_boxplot(fill = "skyblue") +
  labs(
    title = "Distribution of Birth Rates Across Countries",
    x = ""
```

R

```
  theme_minimal()
```

What the boxplot shows

Feature	Meaning
Middle line	Median birth rate
Box	Interquartile range (Q1–Q3)
Whiskers	Typical range
Dots	Outliers

---

3. Optional (Better for your assignment)

A **boxplot per country** (but only for top countries).

```
top_countries <- mean_birthrate %>%
  slice_max(Mean_Birth_Rate, n = 15)
```

```r
ggplot(df_imputed %>%
         filter(`Country Name` %in% top_countries$`Country Name`),
       aes(x = reorder(`Country Name`, Birth_Rate, FUN = mean),
           y = Birth_Rate)) +
  geom_boxplot() +
  coord_flip() +
```

R

```r
    y = "Birth Rate (per 1,000 people)"
  ) +
  theme_minimal()
```

Narrative Interpretation Example

You could write something like:

> The bar chart shows the mean birth rate for each country over the observation period, sorted in descending order. Countries with higher average birth rates appear at the top of the chart, highlighting geographic differences in population growth trends. The box plot summarizes the distribution of birth rates across countries and years, showing the median, interquartile range, and potential outliers. Together, these visualizations provide a clearer overview of birth rate variability across countries compared with plotting individual time series.

✅ Recommendation for your assignment

Use **both**:

- **Bar chart** → compares countries
- **Box plot** → shows statistical distribution

This combination satisfies the requirement for **summary statistics + visualization + interpretation**.

If you want, I can also show you a **very clean "Top 20 countries by mean birth rate" visualization** that will look **much more professional in your report or presentation**.

I already have the summary tables:

```
summary_table <- df_imputed %>%
  group_by( Country Name ) %>%
  summarise(
    Mean_Birth_Rate = mean(Birth_Rate, na.rm = TRUE),
    Min_Birth_Rate = min(Birth_Rate, na.rm = TRUE),
    Max_Birth_Rate = max(Birth_Rate, na.rm = TRUE),
    .groups = "drop"
  )

summary_table |> gt()
```

do the above instruction again

Since you **already created the summary table**, the visualization should now **use** `summary_table` **directly** instead of recalculating statistics from `df_imputed`. This keeps the workflow **clean, reproducible, and aligned with your analysis requirements**.

Your requirement includes:

- **Summary tables with labeled units**
- **Visualizations with titles and axis labels**
- **Narrative interpretation**

Below is the improved workflow.

---

1. Summary Table (Already Correct)

Your code is good. I would only add **units in the column labels** for clarity.

```
library(dplyr)
library(gt)

summary_table <- df_imputed %>%
  group_by(`Country Name`) %>%
  summarise(
    Mean_Birth_Rate = mean(Birth_Rate, na.rm = TRUE),
    Min_Birth_Rate = min(Birth_Rate, na.rm = TRUE),
```

ChatGPT Free offer × Memory full Share •••

```
  )

summary_table |>
  gt() |>
```

```R
    Mean_Birth_Rate = "Mean Birth Rate (per 1,000 people)",
    Min_Birth_Rate = "Minimum Birth Rate",
    Max_Birth_Rate = "Maximum Birth Rate"
  )
```

This produces a **clean formatted summary table**.

---

2. Visualization — Bar Chart of Mean Birth Rates

Since the table already contains the **mean birth rate**, we only need to plot it.

```R
library(ggplot2)

ggplot(summary_table,
       aes(x = reorder(`Country Name`, Mean_Birth_Rate),
           y = Mean_Birth_Rate)) +
  geom_col() +
  coord_flip() +
  labs(
    title = "Mean Birth Rate by Country"
```

```R
  ) +
  theme_minimal()
```

Why this works

- Uses **pre-computed statistics**
- Countries are **sorted by mean**

- `coord_flip()` avoids label crowding

---

3. Optional Visualization — Range of Birth Rates

You can also visualize **min–max variation** using the same table.

```r
ggplot(summary_table,
       aes(x = reorder(`Country Name`, Mean_Birth_Rate),
           ymin = Min_Birth_Rate,
           ymax = Max_Birth_Rate)) +
  geom_linerange() +
  coord_flip() +
  labs(
    title = "Range of Birth Rates by Country",
```

R

```r
  ) +
  theme_minimal()
```

This shows **how much birth rates vary within each country**.

---

4. Narrative Interpretation (Example)

You can include a short explanation like this in your report:

> The summary table reports the mean, minimum, and maximum birth rates for each country over the available time period. The bar chart visualizes the average birth rate across countries, allowing easy comparison of population growth patterns. Countries at the top of the chart exhibit higher average birth rates, while those at the bottom show

relatively lower birth rates. The range plot highlights variability within countries over time, indicating whether birth rates remained stable or experienced significant fluctuations.

✅ Final workflow

1. Clean data → `df_imputed`
2. Create **summary table** → `summary_table`
3. Show table using **gt()**
4. Create **bar chart from summary_table**
5. Add **short narrative interpretation**

If you want, I can also show you a **much better visualization for this dataset (a "Top 20 countries" chart)** because plotting **all countries can still be visually overwhelming** even with bar charts.

Since there are lots of countries, the barchart become almost unreadable. DO you have any solution to this

Yes — this is a **very common problem in country-level datasets**. When there are **150–200 countries**, a full bar chart becomes unreadable. There are several **standard visualization**

solutions used in statistical reports.

Below are the **best 4 approaches**, ranked by usefulness for your case.

---

1. Plot Only the Top and Bottom Countries (Best Option)

Instead of plotting all countries, show **Top 10 and Bottom 10 mean birth rates**.

Code

```r
library(dplyr)
library(ggplot2)

top_bottom <- summary_table %>%
  slice_max(Mean_Birth_Rate, n = 10) %>%
  bind_rows(
    summary_table %>% slice_min(Mean_Birth_Rate, n = 10)
  )

ggplot(top_bottom,
       aes(x = reorder(`Country Name`, Mean_Birth_Rate),
           y = Mean_Birth_Rate)) +
  geom_col() +
  coord_flip() +
```

R

```r
    x = "Country",
    y = "Mean Birth Rate (per 1,000 people)"
  ) +
  theme_minimal()
```

Why this works

- Shows **extreme cases**
- Much **cleaner**
- Often used in **World Bank / UN reports**

---

2. Histogram of Mean Birth Rates (Very Good)

Instead of showing countries individually, show the **distribution**.

Code

```
ggplot(summary_table, aes(x = Mean_Birth_Rate)) +
  geom_histogram(bins = 30) +
  labs(
    title = "Distribution of Mean Birth Rates Across Countries",
    x = "Mean Birth Rate (per 1,000 people)"
```

‹/› R

```
  theme_minimal()
```

What this tells you

- Whether birth rates are **clustered**
- Whether there are **outliers**
- Overall **global pattern**

---

3. Boxplot (Most Compact Summary)

This is the **cleanest statistical visualization**.

```
ggplot(summary_table, aes(y = Mean_Birth_Rate)) +
  geom_boxplot(fill = "skyblue") +
  labs(
    title = "Distribution of Mean Birth Rates Across Countries".
```

‹/› R

```
  theme_minimal()
```

This shows:

- median
- quartiles
- outliers

---

4. Faceted Bar Charts (Advanced Option)

Split countries into **groups of 20–30 per chart**.

```
summary_table <- summary_table %>%
  mutate(group = ceiling(row_number()/25))

ggplot(summary_table,
       aes(x = reorder(`Country Name`, Mean_Birth_Rate),
           y = Mean_Birth_Rate)) +
```

```R
  facet_wrap(~group, scales = "free_y") +
  theme_minimal()
```

This creates **multiple smaller readable charts**.

What I Recommend for Your Assignment

Use **two visuals**:

1️⃣ Histogram (global overview)

2️⃣ Top/Bottom 10 bar chart (country comparison)

This combination is **clear, interpretable, and academically standard**.

Example Narrative

You can write something like:

> Because the dataset contains a large number of countries, displaying all countries in a single bar chart makes interpretation difficult. Instead, a histogram is used to illustrate the overall distribution of mean birth rates across countries. Additionally, a bar chart of the top and bottom countries highlights those with the highest and lowest average birth rates, providing a clearer comparison.

✅ If you want, I can also show you a **very powerful visualization used in demographic research (a world map of birth rates)** — it often **impresses instructors much more than**

bar charts.

Is it possible to create scrollable bar chart?

Yes — but **ggplot2 itself cannot create a scrollable chart** because it produces **static plots**. To make a **scrollable or interactive bar chart**, you need an **interactive visualization library** in R.

The **three best options** are:

1. **plotly (most common)**
2. **ggiraph**
3. **DT interactive tables with bars**

The **best solution for your case is** `plotly`, because it allows **scrolling, zooming, and tooltips** while keeping your `ggplot` code.

Option 1 — Convert Your ggplot to a Scrollable/Zoomable Chart (Recommended)

Code

```
library(ggplot2)
library(plotly)

p <- ggplot(summary_table,
       aes(x = reorder(`Country Name`, Mean_Birth_Rate),
           y = Mean_Birth_Rate)) +
  geom_col() +
  coord_flip() +
  labs(
    title = "Mean Birth Rate by Country",
    x = "Country"
```

‹/› R

```
  theme_minimal()

ggplotly(p)
```

What this gives you

Users can:

- **Scroll**
- **Zoom**
- **Hover to see values**
- **Pan across the chart**

This works very well in **R Markdown / Quarto / HTML output**.

Option 2 — Vertical Scrollable Chart (Plotly Native)

Plotly can create a **tall chart that scrolls vertically**.

```r
library(plotly)

plot_ly(summary_table,
        x = ~Mean_Birth_Rate,
        y = ~`Country Name`,
        type = "bar",
        orientation = "h") %>%
  lavout(
    height = 1500
  )
```

Large height allows the viewer to **scroll through countries**.

Option 3 — Scrollable Table with Bars (Very Professional)

Instead of a chart, you can use a **scrollable table with embedded bar visualization**.

```r
library(DT)
```

This produces a **scrollable interactive table**.

Best Choice for Your Assignment

I recommend:

Interactive Plotly Chart

Because it:

- keeps your **ggplot workflow**
- provides **scroll + zoom**
- looks **professional in reports**

Narrative Example for Your Report

You could write:

> Because the dataset contains a large number of countries, a static bar chart becomes difficult to read. Therefore, an interactive visualization was created using Plotly, allowing users to scroll, zoom, and hover over individual countries to view their mean birth rates. This approach improves readability while still presenting all countries in the dataset.

✅ If you'd like, I can also show you a **very clean "Top 20 + interactive full chart" combination** that is commonly used in **data science reports and academic projects**.

This one worked well:

```
plot_ly(summary_table,
        x = ~Mean_Birth_Rate,
        y = ~ Country Name ,
        type = "bar",
        orientation = "h") %>%
  layout(
    title = "Mean Birth Rate by Country",
    yaxis = list(automargin = TRUE),
    height = 1500
  )
```

But I need a more clean and aesthetic visuals with proper labeling and descending ordering

Great — since the Plotly chart worked, we can **improve it significantly** by adding:

- **Descending sorting**
- **Clear axis labels with units**
- **Better hover labels**
- **Cleaner color styling**
- **Larger margins for readability**
- **Formatted numeric values**

Below is a **clean, publication-quality version**.

Improved Plotly Bar Chart (Clean + Sorted)

Step 1 — Sort the data

R

Step 2 — Create the improved visualization

```r
library(plotly)

plot_ly(
  data = summary_sorted,
  x = ~Mean_Birth_Rate,
  y = ~reorder(`Country Name`, Mean_Birth_Rate),
  type = "bar",
  orientation = "h",
  marker = list(color = "#2C7FB8"),
  hovertemplate = paste(
    "<b>%{y}</b><br>",
    "Mean Birth Rate: %{x:.2f} per 1,000 people",
    "<extra></extra>"
  )
) %>%
  layout(
    title = list(
      text = "Mean Birth Rate by Country",
      x = 0.5
```

```
    ),
    xaxis = list(
      title = "Mean Birth Rate (per 1,000 people)"
```

R

```
      title = "Country",
      automargin = TRUE,
      categoryorder = "total ascending"
    ),
    height = 1500,
    margin = list(l = 150, r = 40, t = 80, b = 60)
  )
```

Improvements Made

1. Descending Order

R

Shows **highest birth rate countries first**.

---

2. Better Hover Information

When hovering:

```
Country Name
Mean Birth Rate: XX per 1,000 people
```

3. Clear Units

Axis labels now include:

```
Birth Rate (per 1,000 people)
```

---

4. Clean Professional Color

```R
```

A **demographic-report style blue**.

---

5. Margin Fix for Long Country Names

```R
```

Prevents labels from being cut off.

Result

The chart will now be:

- **Sorted**
- **Interactive**
- **Readable**
- **Publication-quality**
- **Scrollable**
- **Properly labeled**

Optional (Very Nice Improvement)

Add **mean value labels on bars**:

‹/› R

inside `plot_ly()`.

Example Narrative for Your Report

> An interactive horizontal bar chart was created to visualize the mean birth rate across countries. Countries are sorted in descending order based on their average birth rate. The interactive chart allows users to hover over each country to view precise values, improving readability when dealing with a large number of countries. The visualization clearly highlights differences in average birth rates across countries.

✅ If you'd like, I can also show you a **very impressive visualization used in demographic studies: a "global birth rate trend plot for the top 10 countries"**, which often **scores higher in data analysis assignments**.

Give title and short description for the following step:

```
df_country_stats <- df_tidy %>%
  group_by( Country Name ) %>%
  summarise(
    first_year = min(Year[!is.na(Birth_Rate)]),
    last_year = max(Year[!is.na(Birth_Rate)]),
    total_years = last_year - first_year + 1,
    available_years = sum(!is.na(Birth_Rate)),
    availability_pct = available_years / total_years,
    .groups = "drop"
  )

df_country_stats |> gt()
```

Title:

Country-Level Birth Rate Data Availability Summary

Short Description:

This table summarizes the availability of birth rate data for each country in the dataset. For every country, the first and last years with recorded birth rate values are identified, and the total number of years in that period is calculated. The number of years with available birth

rate data is then counted and used to compute the percentage of data availability. This summary helps evaluate the completeness of each country's data and supports decisions about whether a country should be retained or excluded from further analysis based on the defined availability threshold.

Gice title and description for the following step:

```
valid_countries <- df_country_stats %>%
  filter(availability_pct >= 0.30) %>%
  pull( Country Name )

df_filtered <- df_tidy %>%
  filter( Country Name  %in% valid_countries)

df_filtered |> gt()
```

Title:

Filtering Countries Based on Data Availability Threshold

Short Description:

In this step, countries with sufficient birth rate data are selected for further analysis. Using the previously calculated data availability statistics, only countries with at least **30% of birth rate observations available within their observed time range** are retained. The names of these valid countries are extracted and used to filter the original dataset. The resulting table contains birth rate records only for countries that meet the minimum data

availability threshold, ensuring that subsequent analysis and imputation are performed on datasets with adequate information.

Give me title and secription for the following step:

```
df_imputed <- df_filtered %>%
  group_by( Country Name ) %>%
  arrange(Year) %>%
  mutate(
    Birth_Rate = approx(Year, Birth_Rate, Year, rule = 1)$y
  ) %>%
  fill(Birth_Rate, .direction = "downup") %>%
  ungroup()

df_imputed |> gt()
```

Title:

Imputing Missing Birth Rate Values

Short Description:

This step addresses the remaining missing birth rate values in the filtered dataset using interpolation and directional filling techniques. For each country, the data is first sorted chronologically by year. Missing values within the observed range are then estimated using **linear interpolation**, which calculates intermediate values based on surrounding data points. After interpolation, any remaining missing values at the beginning or end of a

country's time series are filled using **forward and backward filling** (`downup`). This process produces a more complete dataset while preserving the overall trend of birth rates over time for each country.

Give me title and description for the following:

```
summary_table <- df_imputed %>%
  group_by( `Country Name` ) %>%
  summarise(
    Mean_Birth_Rate = mean(Birth_Rate, na.rm = TRUE),
    Min_Birth_Rate = min(Birth_Rate, na.rm = TRUE),
    Max_Birth_Rate = max(Birth_Rate, na.rm = TRUE),
    .groups = "drop"
  )


summary_sorted <- summary_table %>%
  arrange(desc(Mean_Birth_Rate))

summary_sorted |>
  gt() |>
  cols_label(
    `Country Name` = "Country",
    Mean_Birth_Rate = "Mean Birth Rate (per 1,000
people)",
    Min_Birth_Rate = "Minimum Birth Rate",
```

```
  Max_Birth_Rate = "Maximum Birth Rate"
)
```

Title:

Summary Statistics of Imputed Birth Rates by Country

Short Description:

This step calculates key summary statistics for the imputed birth rate dataset. For each country, the **mean, minimum, and maximum birth rates** are computed across all years. The results are then sorted in descending order of mean birth rate to highlight countries with the highest average values. The table is formatted with clear labels and units, providing an easily interpretable overview of birth rate trends and variability across countries. This summary supports both visual analysis and comparison between countries.

Givwe title and give short description of the following step:

```
plot_ly(
  data = summary_sorted,
  x = ~Mean_Birth_Rate,
  y = ~reorder( Country Name , Mean_Birth_Rate),
  type = "bar",
  orientation = "h",
  marker = list(color = "#2C7FB8"),
  hovertemplate = paste(
    "<b>%{y}</b><br>",
```

```
    "Mean Birth Rate: %{x:.2f} per 1,000 people",
    "<extra></extra>"
  )
) %>%
  layout(
    title = list(
      text = "Mean Birth Rate by Country",
      x = 0.5
    ),
    xaxis = list(
      title = "Mean Birth Rate (per 1,000 people)"
    ),
    yaxis = list(
      title = "Country",
      automargin = TRUE,
      categoryorder = "total ascending"
    ),
    height = 1500,
    margin = list(l = 150, r = 40, t = 80, b = 60)
  )
```

Title:

Interactive Horizontal Bar Chart of Mean Birth Rates by Country

Short Description:

This step visualizes the mean birth rates across countries using an interactive horizontal bar chart. Countries are sorted in descending order of mean birth rate, and each bar is color-coded for clarity. Hovering over a bar displays the country name and its exact mean

birth rate per 1,000 people. The interactive chart allows scrolling and panning to accommodate the large number of countries, making it easy to compare birth rates and identify countries with the highest and lowest averages.

Give a short conclusion

Conclusion:
The analysis provides a comprehensive overview of global birth rate trends. After filtering countries with insufficient data and imputing missing values, summary statistics revealed differences in mean, minimum, and maximum birth rates across countries. The interactive bar chart highlights countries with the highest and lowest average birth rates, making comparisons straightforward. Overall, the workflow ensures reliable, complete, and interpretable data for understanding global birth rate patterns.

give APA citation for this transcript

Here’s a proper APA-style citation for this chat transcript:

APA (7th edition):

OpenAI. (2026, March 8). *ChatGPT conversation with K. M. Qaiduzzaman on birth rate data analysis in R*. OpenAI. https://chat.openai.com/ ↗

If you want, I can also provide a **short in-text citation version** suitable for your report.